

and Liu [180], Golub and Van Loan [192], Press, Teukolsky, Vetterling, and Flannery [365, 366], and Strang [422, 423].

Golub and Van Loan [192] discuss numerical stability. They show why $\det(A)$ is not necessarily a good indicator of the stability of a matrix A , proposing instead to use $\|A\|_\infty \|A^{-1}\|_\infty$, where $\|A\|_\infty = \max \{\sum_{j=1}^n |a_{ij}| : 1 \leq i \leq n\}$. They also address the question of how to compute this value without actually computing A^{-1} .

Gaussian elimination, upon which the LU and LUP decompositions are based, was the first systematic method for solving linear systems of equations. It was also one of the earliest numerical algorithms. Although it was known earlier, its discovery is commonly attributed to C. F. Gauss (1777–1855). In his famous paper [424], Strassen showed that an $n \times n$ matrix can be inverted in $O(n^{\lg 7})$ time. Winograd [460] originally proved that matrix multiplication is no harder than matrix inversion, and the converse is due to Aho, Hopcroft, and Ullman [5].

Another important matrix decomposition is the *singular value decomposition*, or *SVD*. The SVD factors an $m \times n$ matrix A into $A = Q_1 \Sigma Q_2^T$, where Σ is an $m \times n$ matrix with nonzero values only on the diagonal, Q_1 is $m \times m$ with mutually orthonormal columns, and Q_2 is $n \times n$, also with mutually orthonormal columns. Two vectors are *orthonormal* if their inner product is 0 and each vector has a norm of 1. The books by Strang [422, 423] and Golub and Van Loan [192] contain good treatments of the SVD.

Strang [423] has an excellent presentation of symmetric positive-definite matrices and of linear algebra in general.

Many problems take the form of maximizing or minimizing an objective, given limited resources and competing constraints. If you can specify the objective as a linear function of certain variables, and if you can specify the constraints on resources as equalities or inequalities on those variables, then you have a *linear-programming problem*. Linear programs arise in a variety of practical applications. We begin by studying an application in electoral politics.

A political problem

Suppose that you are a politician trying to win an election. Your district has three different types of areas—urban, suburban, and rural. These areas have, respectively, 100,000, 200,000, and 50,000 registered voters. Although not all the registered voters actually go to the polls, you decide that to govern effectively, you would like at least half the registered voters in each of the three regions to vote for you. You are honorable and would never consider supporting policies you don't believe in. You realize, however, that certain issues may be more effective in winning votes in certain places. Your primary issues are preparing for a zombie apocalypse, equipping sharks with lasers, building highways for flying cars, and allowing dolphins to vote.

According to your campaign staff's research, you can estimate how many votes you win or lose from each population segment by spending \$1,000 on advertising on each issue. This information appears in the table of Figure 29.1. In this table, each entry indicates the number of thousands of either urban, suburban, or rural voters who would be won over by spending \$1,000 on advertising in support of a particular issue. Negative entries denote votes that would be lost. Your task is to figure out the minimum amount of money that you need to spend in order to win 50,000 urban votes, 100,000 suburban votes, and 25,000 rural votes.

You could, by trial and error, devise a strategy that wins the required number of votes, but the strategy you come up with might not be the least expensive one. For example, you could devote \$20,000 of advertising to preparing for a zombie

policy	urban	suburban	rural
zombie apocalypse	-2	5	3
sharks with lasers	8	2	-5
highways for flying cars	0	0	10
dolphins voting	10	0	-2

Figure 29.1 The effects of policies on voters. Each entry describes the number of thousands of urban, suburban, or rural voters who could be won over by spending \$1,000 on advertising support of a policy on a particular issue. Negative entries denote votes that would be lost.

apocalypse, \$0 to equipping sharks with lasers, \$4,000 to building highways for flying cars, and \$9,000 to allowing dolphins to vote. In this case, you would win $(20 \cdot -2) + (0 \cdot 8) + (4 \cdot 0) + (9 \cdot 10) = 50$ thousand urban votes, $(20 \cdot 5) + (0 \cdot 2) + (4 \cdot 0) + (9 \cdot 0) = 100$ thousand suburban votes, and $(20 \cdot 3) + (0 \cdot -5) + (4 \cdot 10) + (9 \cdot -2) = 82$ thousand rural votes. You would win the exact number of votes desired in the urban and suburban areas and more than enough votes in the rural area. (In fact, according to your model, in the rural area you would receive more votes than there are voters.) In order to garner these votes, you would have paid for $20 + 0 + 4 + 9 = 33$ thousand dollars of advertising.

It's natural to wonder whether this strategy is the best possible. That is, can you achieve your goals while spending less on advertising? Additional trial and error might help you to answer this question, but a better approach is to formulate (or *model*) this question mathematically.

The first step is to decide what decisions you have to make and to introduce variables that capture these decisions. Since you have four decisions, you introduce four *decision variables*:

- x_1 is the number of thousands of dollars spent on advertising on preparing for a zombie apocalypse,
- x_2 is the number of thousands of dollars spent on advertising on equipping sharks with lasers,
- x_3 is the number of thousands of dollars spent on advertising on building highways for flying cars, and
- x_4 is the number of thousands of dollars spent on advertising on allowing dolphins to vote.

You then think about *constraints*, which are limits, or restrictions, on the values that the decision variables can take. You can write the requirement that you win at least 50,000 urban votes as

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50. \quad (29.1)$$

Similarly, you can write the requirements that you win at least 100,000 suburban votes and 25,000 rural votes as

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.2)$$

and

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25. \quad (29.3)$$

Any setting of the variables x_1, x_2, x_3, x_4 that satisfies inequalities (29.1)–(29.3) yields a strategy that wins a sufficient number of each type of vote.

Finally, you think about your *objective*, which is the quantity that you wish to either minimize or maximize. In order to keep costs as small as possible, you would like to minimize the amount spent on advertising. That is, you want to minimize the expression

$$x_1 + x_2 + x_3 + x_4. \quad (29.4)$$

Although negative advertising often occurs in political campaigns, there is no such thing as negative-cost advertising. Consequently, you require that

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, \text{ and } x_4 \geq 0. \quad (29.5)$$

Combining inequalities (29.1)–(29.3) and (29.5) with the objective of minimizing (29.4) produces what is known as a “linear program.” We can format this problem tabularly as

$$\text{minimize} \quad x_1 + x_2 + x_3 + x_4 \quad (29.6)$$

subject to

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \quad (29.7)$$

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.8)$$

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \quad (29.9)$$

$$x_1, x_2, x_3, x_4 \geq 0. \quad (29.10)$$

The solution to this linear program yields your optimal strategy.

The remainder of this chapter covers how to formulate linear programs and is an introduction to modeling in general. Modeling refers to the general process of converting a problem into a mathematical form amenable to solution by an algorithm. Section 29.1 discusses briefly the algorithmic aspects of linear programming, although it does not include the details of a linear-programming algorithm. Throughout this book, we have seen ways to model problems, such as by shortest paths and connectivity in a graph. When modeling a problem as a linear program, you go through the steps used in this political example—identifying the decision variables, specifying the constraints, and formulating the objective function. In order to model a problem as a linear program, the constraints and objectives must be

linear. In Section 29.2, we will see several other examples of modeling via linear programs. Section 29.3 discusses duality, an important concept in linear programming and other optimization algorithms.

29.1 Linear programming formulations and algorithms

Linear programs take a particular form, which we will examine in this section. Multiple algorithms have been developed to solve linear programs. Some run in polynomial time, some do not, but they are all too complicated to show here. Instead, we will give an example that demonstrates some ideas behind the simplex algorithm, which is currently the most commonly deployed solution method.

General linear programs

In the general linear-programming problem, we wish to optimize a linear function subject to a set of linear inequalities. Given a set of real numbers $a_1, a_2, \dots, a_n$ and a set of variables $x_1, x_2, \dots, x_n$, we define a *linear function* f on those variables by

$$f(x_1, x_2, \dots, x_n) = a_1x_1 + a_2x_2 + \dots + a_nx_n = \sum_{j=1}^n a_jx_j .$$

If b is a real number and f is a linear function, then the equation

$$f(x_1, x_2, \dots, x_n) = b$$

is a *linear equality* and the inequalities

$$f(x_1, x_2, \dots, x_n) \leq b \text{ and } f(x_1, x_2, \dots, x_n) \geq b$$

are *linear inequalities*. We use the general term *linear constraints* to denote either linear equalities or linear inequalities. Linear programming does not allow strict inequalities. Formally, a *linear-programming problem* is the problem of either minimizing or maximizing a linear function subject to a finite set of linear constraints. If minimizing, we call the linear program a *minimization linear program*, and if maximizing, we call the linear program a *maximization linear program*.

In order to discuss linear-programming algorithms and properties, it will be helpful to use a standard notation for the input. By convention, a maximization linear program takes as input n real numbers $c_1, c_2, \dots, c_n$; m real numbers $b_1, b_2, \dots, b_m$; and mn real numbers a_{ij} for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$.

The goal is to find n real numbers $x_1, x_2, \dots, x_n$ that

$$\text{maximize } \sum_{j=1}^n c_j x_j \quad (29.11)$$

subject to

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \text{ for } i = 1, 2, \dots, m \quad (29.12)$$

$$x_j \geq 0 \text{ for } j = 1, 2, \dots, n . \quad (29.13)$$

We call expression (29.11) the **objective function** and the $n + m$ inequalities in lines (29.12) and (29.13) the **constraints**. The n constraints in line (29.13) are the **nonnegativity constraints**. It can sometimes be more convenient to express a linear program in a more compact form. If we create an $m \times n$ matrix $A = (a_{ij})$, an m -vector $b = (b_i)$, an n -vector $c = (c_j)$, and an n -vector $x = (x_j)$, then we can rewrite the linear program defined in (29.11)–(29.13) as

$$\text{maximize } c^T x \quad (29.14)$$

subject to

$$Ax \leq b \quad (29.15)$$

$$x \geq 0 . \quad (29.16)$$

In line (29.14), $c^T x$ is the inner product of two n -vectors. In inequality (29.15), Ax is the m -vector that is the product of an $m \times n$ matrix and an n -vector, and in inequality (29.16), $x \geq 0$ means that each entry of the vector x must be nonnegative. We call this representation the **standard form** for a linear program, and we adopt the convention that A , b , and c always have the dimensions given above.

The standard form above may not naturally correspond to real-life situations you are trying to model. For example, you might have equality constraints or variables that can take on negative values. Exercises 29.1-6 and 29.1-7 ask you to show how to convert any linear program into this standard form.

We now introduce terminology to describe solutions to linear programs. We denote a particular setting of the values in a variable, say x , by putting a bar over the variable name: $\bar{x}$. If $\bar{x}$ satisfies all the constraints, then it is a **feasible solution**, but if it fails to satisfy at least one constraint, then it is an **infeasible solution**. We say that a solution $\bar{x}$ has **objective value** $c^T \bar{x}$. A feasible solution $\bar{x}$ whose objective value is maximum over all feasible solutions is an **optimal solution**, and we call its objective value $c^T \bar{x}$ the **optimal objective value**. If a linear program has no feasible solutions, we say that the linear program is **infeasible**, and otherwise, it is **feasible**. The set of points that satisfy all the constraints is the **feasible region**. If a linear program has some feasible solutions but does not have a finite optimal objective value, then the feasible region is **unbounded** and so is the linear program. Exercise 29.1-5 asks you to show that a linear program can have a finite optimal objective value even if the feasible region is unbounded.

One of the reasons for the power and popularity of linear programming is that linear programs can, in general, be solved efficiently. There are two classes of algorithms, known as ellipsoid algorithms and interior-point algorithms, that solve linear programs in polynomial time. In addition, the simplex algorithm is widely used. Although it does not run in polynomial time in the worst case, it tends to perform well in practice.

We will not give a detailed algorithm for linear programming, but will discuss a few important ideas. First, we will give an example of using a geometric procedure to solve a two-variable linear program. Although this example does not immediately generalize to an efficient algorithm for larger problems, it introduces some important concepts for linear programming and for optimization in general.

A two-variable linear program

Let us first consider the following linear program with two variables:

$$\text{maximize } x_1 + x_2 \quad (29.17)$$

subject to

$$4x_1 - x_2 \leq 8 \quad (29.18)$$

$$2x_1 + x_2 \leq 10 \quad (29.19)$$

$$5x_1 - 2x_2 \geq -2 \quad (29.20)$$

$$x_1, x_2 \geq 0. \quad (29.21)$$

Figure 29.2(a) graphs the constraints in the (x_1, x_2) -Cartesian coordinate system. The feasible region in the two-dimensional space (highlighted in blue in the figure) is convex.¹ Conceptually, you could evaluate the objective function $x_1 + x_2$ at each point in the feasible region, and then identify a point that has the maximum objective value as an optimal solution. For this example (and for most linear programs), however, the feasible region contains an infinite number of points, and so to solve this linear program, you need an efficient way to find a point that achieves the maximum objective value without explicitly evaluating the objective function at every point in the feasible region.

In two dimensions, you can optimize via a graphical procedure. The set of points for which $x_1 + x_2 = z$, for any z , is a line with a slope of -1 . Plotting $x_1 + x_2 = 0$ produces the line with slope -1 through the origin, as in Figure 29.2(b). The intersection of this line and the feasible region is the set of feasible solutions that have an objective value of 0. In this case, that intersection of the line with the feasible region is the single point $(0, 0)$. More generally, for any value z , the

¹ An intuitive definition of a convex region is that it fulfills the requirement that for any two points in the region, all points on a line segment between them are also in the region.

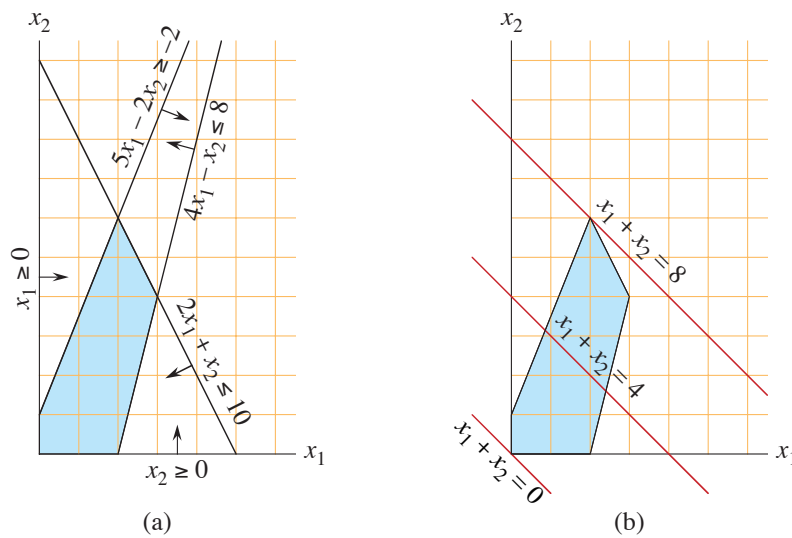


Figure 29.2 (a) The linear program given in (29.18)–(29.21). Each constraint is represented by a line and a direction. The intersection of the constraints, which is the feasible region, is highlighted in blue. (b) The red lines show, respectively, the points for which the objective value is 0, 4, and 8. The optimal solution to the linear program is $x_1 = 2$ and $x_2 = 6$ with objective value 8.

intersection of the line $x_1 + x_2 = z$ and the feasible region is the set of feasible solutions that have objective value z . Figure 29.2(b) shows the lines $x_1 + x_2 = 0$, $x_1 + x_2 = 4$, and $x_1 + x_2 = 8$. Because the feasible region in Figure 29.2 is bounded, there must be some maximum value z for which the intersection of the line $x_1 + x_2 = z$ and the feasible region is nonempty. Any point in the feasible region that maximizes $x_1 + x_2$ is an optimal solution to the linear program, which in this case is the vertex of the feasible region at $x_1 = 2$ and $x_2 = 6$, with objective value 8.

It is no accident that an optimal solution to the linear program occurs at a vertex of the feasible region. The maximum value of z for which the line $x_1 + x_2 = z$ intersects the feasible region must be on the boundary of the feasible region, and thus the intersection of this line with the boundary of the feasible region is either a single vertex or a line segment. If the intersection is a single vertex, then there is just one optimal solution, and it is that vertex. If the intersection is a line segment, every point on that line segment must have the same objective value. In particular, both endpoints of the line segment are optimal solutions. Since each endpoint of a line segment is a vertex, there is an optimal solution at a vertex in this case as well.

Although you cannot easily graph linear programs with more than two variables, the same intuition holds. If you have three variables, then each constraint corresponds to a half-space in three-dimensional space. The intersection of these half-

spaces forms the feasible region. The set of points for which the objective function obtains a given value z is now a plane (assuming no degenerate conditions). If all coefficients of the objective function are nonnegative, and if the origin is a feasible solution to the linear program, then as you move this plane away from the origin, in a direction normal to the objective function, you find points of increasing objective value. (If the origin is not feasible or if some coefficients in the objective function are negative, the intuitive picture becomes slightly more complicated.) As in two dimensions, because the feasible region is convex, the set of points that achieve the optimal objective value must include a vertex of the feasible region. Similarly, if you have n variables, each constraint defines a half-space in n -dimensional space. We call the feasible region formed by the intersection of these half-spaces a *simplex*. The objective function is now a hyperplane and, because of convexity, an optimal solution still occurs at a vertex of the simplex. Any algorithm for linear programming must also identify linear programs that have no solutions, as well as linear programs that have no finite optimal solution.

The *simplex algorithm* takes as input a linear program and returns an optimal solution. It starts at some vertex of the simplex and performs a sequence of iterations. In each iteration, it moves along an edge of the simplex from a current vertex to a neighboring vertex whose objective value is no smaller than that of the current vertex (and usually is larger.) The simplex algorithm terminates when it reaches a local maximum, which is a vertex from which all neighboring vertices have a smaller objective value. Because the feasible region is convex and the objective function is linear, this local optimum is actually a global optimum. In Section 29.3, we'll see an important concept called "duality," which we'll use to prove that the solution returned by the simplex algorithm is indeed optimal.

The simplex algorithm, when implemented carefully, often solves general linear programs quickly in practice. With some carefully contrived inputs, however, the simplex algorithm can require exponential time. The first polynomial-time algorithm for linear programming was the *ellipsoid algorithm*, which runs slowly in practice. A second class of polynomial-time algorithms are known as *interior-point methods*. In contrast to the simplex algorithm, which moves along the exterior of the feasible region and maintains a feasible solution that is a vertex of the simplex at each iteration, these algorithms move through the interior of the feasible region. The intermediate solutions, while feasible, are not necessarily vertices of the simplex, but the final solution is a vertex. For large inputs, interior-point algorithms can run as fast as, and sometimes faster than, the simplex algorithm. The chapter notes point you to more information about these algorithms.

If you add to a linear program the additional requirement that all variables take on integer values, you have an *integer linear program*. Exercise 34.5-3 on page 1098 asks you to show that just finding a feasible solution to this problem is NP-hard. Since no polynomial-time algorithms are known for any NP-hard prob-

lems, there is no known polynomial-time algorithm for integer linear programming. In contrast, a general linear-programming problem can be solved in polynomial time.

Exercises

29.1-1

Consider the linear program

minimize $-2x_1 + 3x_2$

subject to

$$x_1 + x_2 = 7$$

$$x_1 - 2x_2 \leq 4$$

$$x_1 \geq 0.$$

Give three feasible solutions to this linear program. What is the objective value of each one?

29.1-2

Consider the following linear program, which has a nonpositivity constraint:

minimize $2x_1 + 7x_2 + x_3$

subject to

$$x_1 - x_3 = 7$$

$$3x_1 + x_2 \geq 24$$

$$x_2 \geq 0$$

$$x_3 \leq 0.$$

Give three feasible solutions to this linear program. What is the objective value of each one?

29.1-3

Show that the following linear program is infeasible:

maximize $3x_1 - 2x_2$

subject to

$$x_1 + x_2 \leq 2$$

$$-2x_1 - 2x_2 \leq -10$$

$$x_1, x_2 \geq 0.$$